The first step towards a (pseudo)implementation for our data structures is, as always, presenting a pseudo-class what models it; in this case, to model an SS-tree we are going to need a class modeling tree nodes; once we build an SS-tree through its nodes, to access it we just need a pointer to the tree's root. For convenience, as shown in listing 10.1, we will include this link and the values for parameters `m` and `M` into the `SsTree` class, as well as the dimensionality `k` of each data entry, and assume all these values are available from each of the tree nodes.

As we have seen, SS-trees (like R-trees) have two different kind of nodes, leaves and internal nodes, that are structurally and behaviorally different: the former stores k-dimensional tuples (references to the points in our dataset), the latter has only links to its children (which are also nodes of the tree).

To keep things simple and as much as possible language-agnostic, we will store both an array of children and an array of points into each node, and a boolean flag will tell apart leaves from internal nodes: the `children` array will be empty for leaves and the `points` array will be empty for internal nodes.

Listing 10.1 The SsTree and SsNode classes

```
class SsNode
  #type tuple(k)
  centroid
  #type float
  radius
```